

PartnerAPI

UML Design Package

Software Engineering Project

October 6, 2026

Contents

1	Architecture Overview	2
2	Use Case Diagram	2
3	Class Diagram	3
4	Sequence Diagram	4
5	Activity Diagram	5
6	Booking State Diagram	6
7	Cohesion	6
8	Coupling	7
9	Design Rationale	7
10	Conclusion	7

1 Architecture Overview

PartnerAPI separates the public partner interface from the internal booking platform.

The architecture follows the principle:

“Public API clients should depend on stable public contracts rather than internal booking-system implementation details.”

This separation reduces coupling between external partner integrations and the internal booking-system implementation. It also allows the internal booking platform to evolve without requiring unnecessary changes to partner integrations.

2 Use Case Diagram

The use case model identifies the primary interactions between partner agencies, administrators, and the PartnerAPI platform.

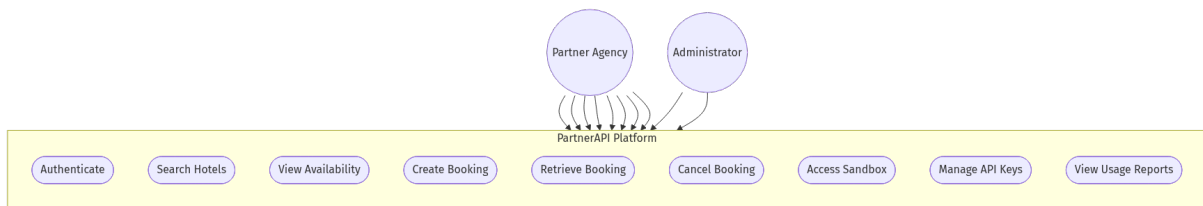


Figure 1: PartnerAPI Use Case Diagram

The primary use cases represented in the system include:

- Authenticate
- Search Hotels
- View Availability
- Create Booking
- Retrieve Booking
- Cancel Booking
- Access Sandbox
- Manage API Keys
- View Usage Reports

These use cases represent the principal interactions required by external partner agencies and administrative users.

3 Class Diagram

The class diagram represents the primary domain objects and their relationships within the PartnerAPI system.

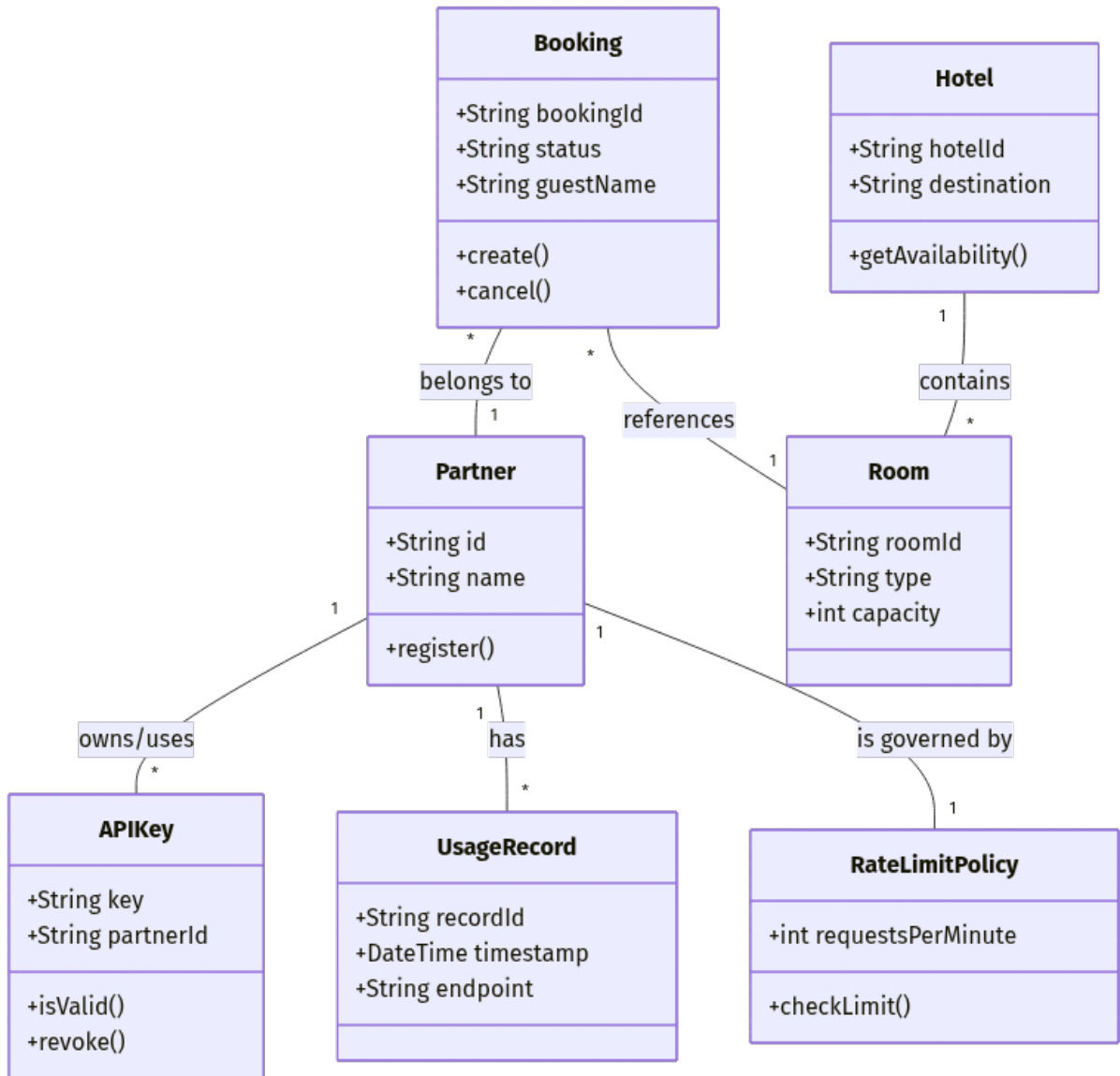


Figure 2: PartnerAPI Class Diagram

The key domain classes include:

- **Partner** – represents a registered partner agency.
- **APIKey** – represents credentials used to authenticate API requests.
- **Hotel** – represents a hotel available through the booking platform.
- **Room** – represents a bookable room associated with a hotel.
- **Booking** – represents a partner-created hotel reservation.

- **RateLimitPolicy** – defines request-rate restrictions for API consumers.
- **UsageRecord** – records API usage for monitoring and reporting.

4 Sequence Diagram

The sequence diagram describes the “Search and Book a Hotel Through the API” scenario.

The interaction begins when a partner agency submits a request to the public API. The request is authenticated and checked against the applicable rate-limit policy before hotel and booking services process the request.

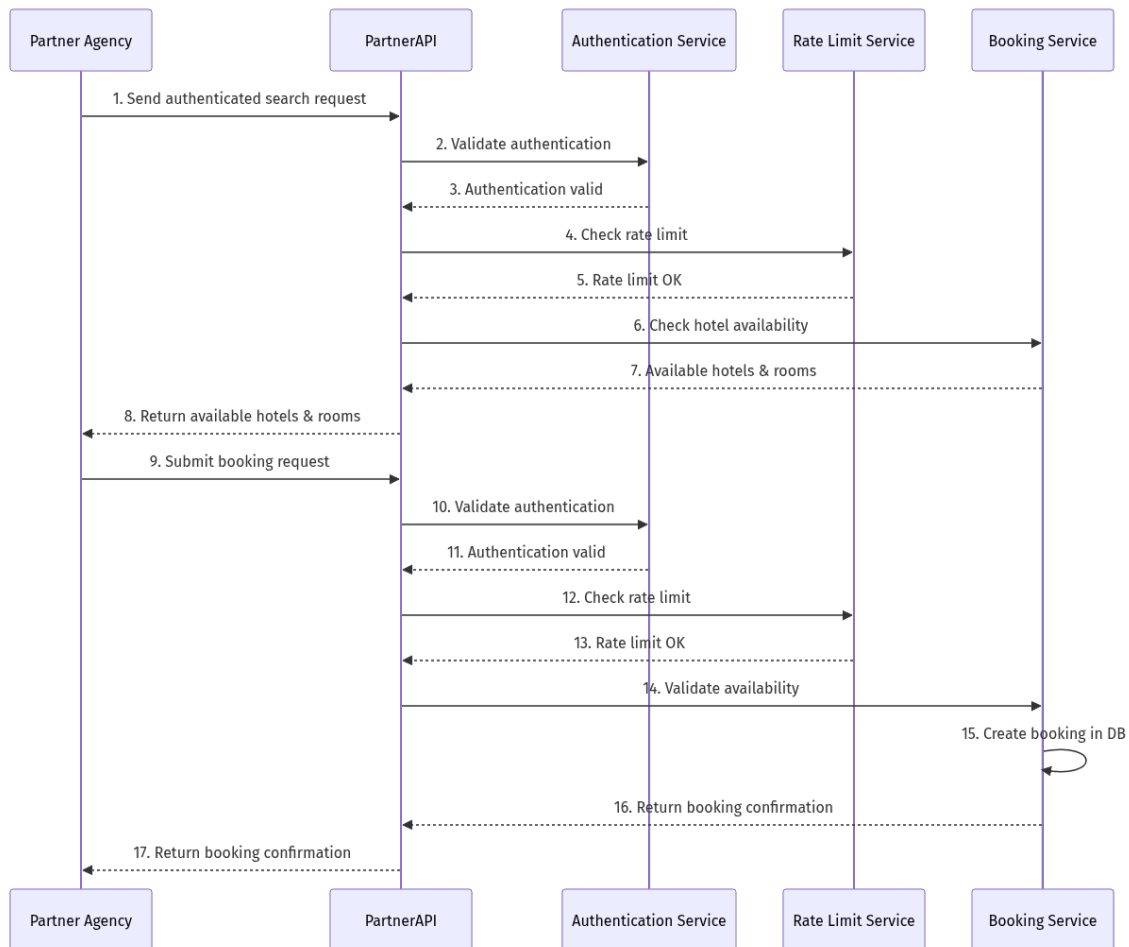


Figure 3: Search and Book Hotel Sequence

The sequence illustrates the separation between the external partner client, the public API layer, authentication and rate-limit services, and the internal hotel and booking services.

5 Activity Diagram

The activity diagram details the partner booking workflow from authentication through booking confirmation.

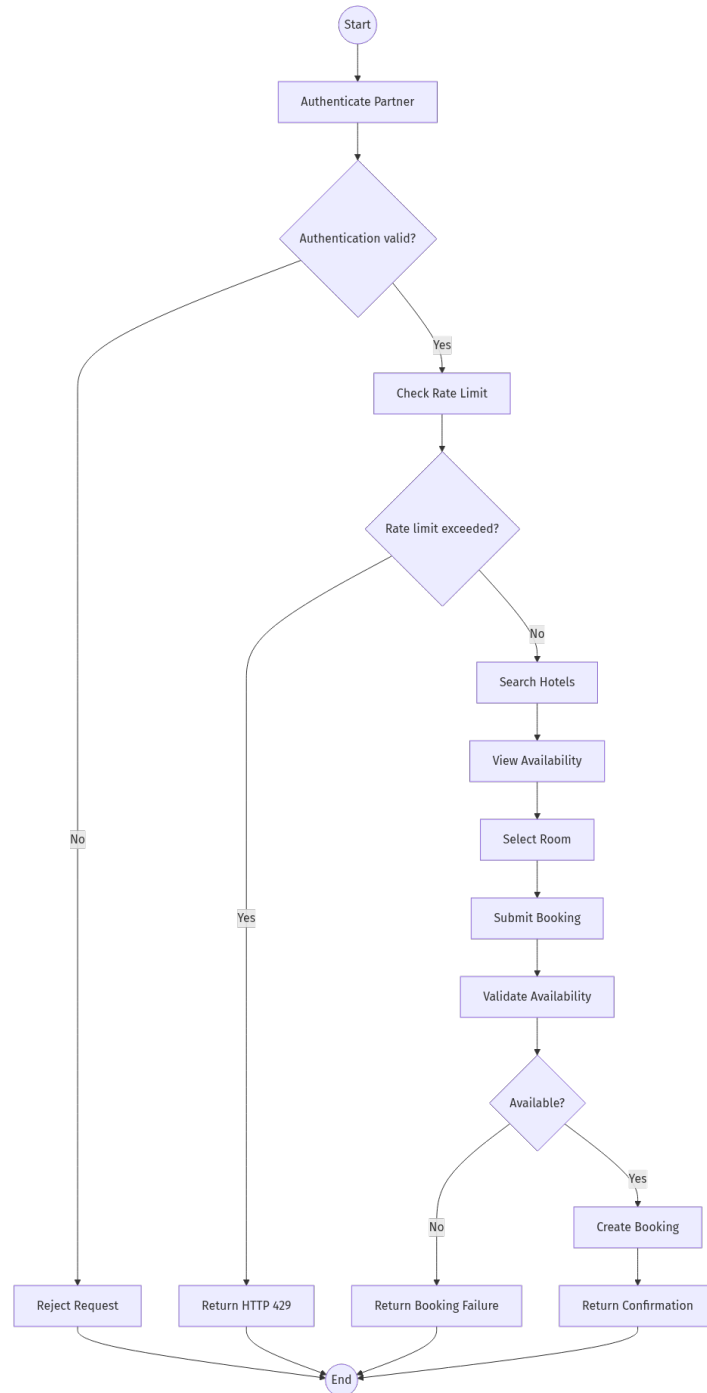


Figure 4: Partner Booking Activity

The workflow includes authentication, rate-limit validation, hotel search, room selection, availability validation, booking creation, and confirmation.

6 Booking State Diagram

A booking progresses through controlled states during its lifecycle.

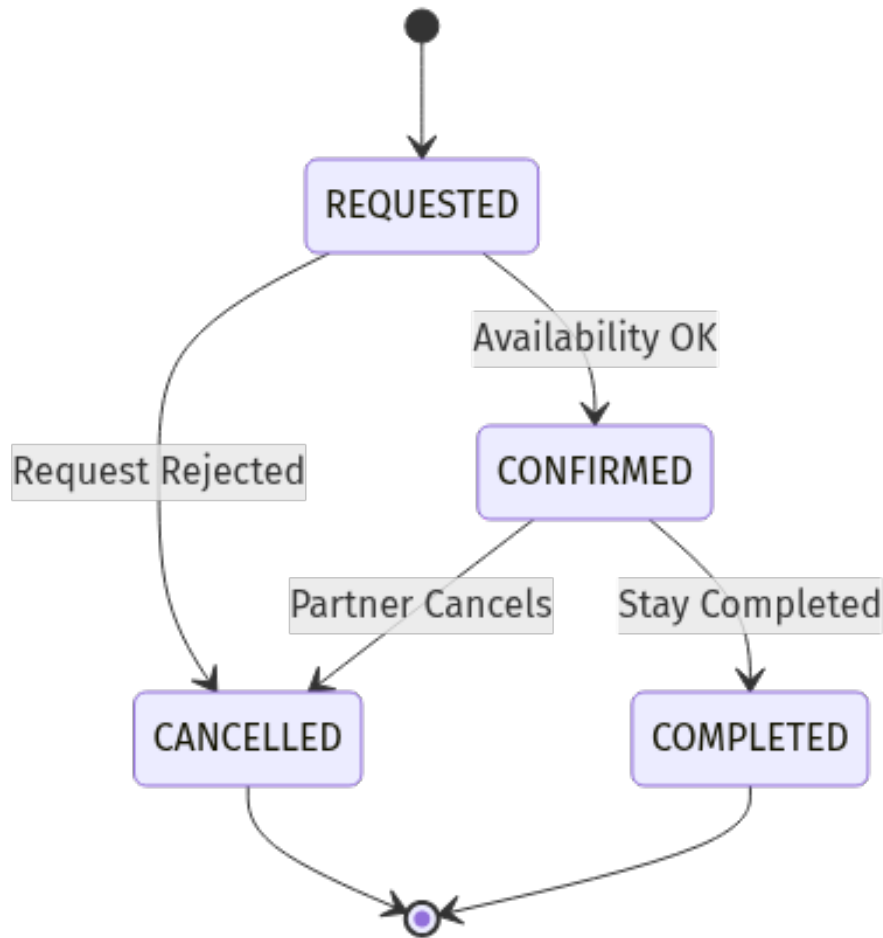


Figure 5: PartnerAPI Booking State Diagram

The state model provides a controlled representation of booking lifecycle transitions. It helps ensure that booking operations are performed only when valid state transitions are permitted.

7 Cohesion

The system is organised into modules with focused responsibilities. This promotes high cohesion because each module primarily addresses one area of system functionality.

- **Authentication module** handles authentication and credential validation.
- **Booking module** handles booking lifecycle logic.
- **Rate-limit module** handles API request limits.

- **Reporting module** handles usage and reporting data.
- **Partner module** handles partner configuration and related management operations.

Each module therefore has a focused responsibility, reducing unnecessary dependencies between unrelated areas of the system.

8 Coupling

The public API is separated from the internal booking implementation.

The API communicates through stable service interfaces rather than directly depending on internal database structures. This reduces coupling between the external partner integrations and the internal booking platform.

As a result, the internal booking platform can evolve without forcing partners to modify their integrations unnecessarily, provided that the public API contracts remain stable.

9 Design Rationale

The UML design follows several software engineering principles:

- **High cohesion** – each module has a focused responsibility.
- **Low coupling** – external partners depend on stable API contracts rather than internal implementation details.
- **Stable API contracts** – public interfaces are separated from internal booking implementation.
- **Clear responsibility boundaries** – authentication, booking, rate limiting, partner management, and reporting have distinct responsibilities.
- **Independent testing** – separated components can be tested with focused test cases.
- **Controlled change** – changes to internal implementation can be made with reduced impact on external partner integrations.

10 Conclusion

The UML design package defines the architecture and behavioural structure of PartnerAPI through use case, class, sequence, activity, and state diagrams.

The design separates the public API from the internal booking platform while maintaining clear responsibility boundaries. High cohesion within modules and low coupling between external partners and internal implementation support maintainability, independent testing, and controlled system evolution.

Overall, the UML models provide a consistent structural and behavioural representation of the PartnerAPI system and support the project's objective of enabling partner agencies to integrate with the hotel booking platform through a stable public API.